Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O objetivo da atividade é desenvolver um programa com Linguagem C que simule o funcionamento do embarque de passageiros nos carros de metrô. A proposta da atividade é garantir que o embarque aconteça de forma organizada e respeitando a quantidade de assentos disponíveis e as condições necessárias para a partida dos carros de metrô.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Implementar station_init	OK	Entrar em sync_car.c e ver a inicialização do mutex, das duas variáveis de condição e das variáveis de controle da struct station
Implementar station_load_car	OK	Executar ./sync_car e ver que o carro espera concluir o embarque antes de sair
Implementar station_wait_for_car	OK	Entrar em sync_car.c e ver que a função bloqueia a thread com pthread_cond_wait enquanto não tem carros na estação ou assentos disponíveis
Implementar station_on_board	OK	Entrar em sync_car.c e ver que a função atualiza a quantidade de passageiros que confirmaram o embarque e sinaliza o carro quando as condições de partida são atendidas
Não utilizar semáforos	OK	Analisar sync_car.c e sync_car.h e ver que a sincronização é feita com mutex e variáveis de condição e sem usar de semáforos

Permitir que passageiros cheguem durante o embarque, caso ainda existam assentos	OK	Executar o teste adicional de chegada tardia e verificar se um passageiro que chega durante o embarque consegue obter um assento no mesmo carro, desde que ainda existam vagas
Não utilizar busy wait	OK	Entrar em sync_car.c e ver que as threads esperam usando pthread_cond_wait sem usar laços de espera contínua para verificar condições
Gerar o executável sync_car utilizando o Makefile	OK	Enviar o comando make no terminal e ver que o executável é gerado
Makefile	OK	Enviar os comandos make, make clean e ./sync_car no terminal

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

sync_car.c: Implementa as quatro funções responsáveis pela sincronização entre passageiros e carros.

sync_car.h: Define a estrutura station e declara as funções utilizadas no projeto.

sync_car-runner.c: Arquivo de testes fornecido pelo professor, responsável por verificar o funcionamento da implementação.

makefile: Define os comandos e as opções necessárias para compilar o programa.

gitignore: Define os arquivos que devem ser ignorados pelo Git, incluindo o executável gerado.

Decisões técnicas:

1. utilizar um único mutex e duas variáveis de condição para controlar a comunicação entre as threads. Dessa forma, é possível proteger as informações compartilhadas e organizar separadamente a espera dos passageiros pela chegada do carro e a espera do carro pela conclusão do embarque.

2. Usar contadores separados para controlar os passageiros que estão esperando por uma vaga, os assentos disponíveis e quem já reservou um assento mas ainda não confirmou o embarque. Fiz isso porque reservar um assento e confirmar o embarque são coisas diferentes. Assim, da pra acompanhar corretamente o estado dos passageiros e evitar que o carro saia antes da hora.

3. Usei pthread_cond_wait, pthread_cond_broadcast e pthread_cond_signal pra controlar quando as threads devem esperar e quando podem continuar. Assim é possível avisar os passageiros quando o carro chega e avisar ao carro quando as condições para partir são atendidas sem precisar ficar verificando as condições.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	**Nome curto**	**Contexto**	**Motivo da troca (se houve)**
S1	Desenvolvimento geral	Comecei criando os arquivos e declarando as funções da atividade e a struct station. Depois fiz a implementação das funções dentro do arquivo sync_car.c	Tive que mudar de estratégia e fazer ajustes na sincronização, pois havia um problema de lógica na parte em que os passageiros faziam reserva.
S2	Ajustes na sincronização	Revisei a lógica de comunicação entre passageiros e carro e fiz ajustes no controle do embarque para consertar a lógica do código.	Garantir que os passageiros conseguissem reservar assentos corretamente e que o carro aguardasse as confirmações antes de partir.
S3	Testes e validação	Compilei e executei o programa utilizando o teste fornecido na atividade	Verificar se a implementação estava funcionando corretamente e que atendia todos requisitos

5.2 Diário de Tentativas

#	Estrat	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Criei a estrutura inicial do projeto e organizei as declarações necessárias no arquivo sync_car.h.	OK	Não teve falha	No início do desenvolvimento, na organização dos arquivos e da estrutura da atividade.	Commit e36a8cc.
2	S1	Fiz o Makefile para compilar os arquivos do projeto e gerar o executável sync_car.	OK	Não teve falha	Depois de fazer a estrutura inicial, na preparação da compilação do projeto.	Commit 5f837c2.
3	S1	Adicionei ao projeto o arquivo de teste fornecido na atividade para verificar o funcionamento da implementação.	OK	Não teve falha	Depois de fazer o Makefile, antes de começar a implementação das funções.	Commit 0d44032.
4	S1	Implementei station_load_car para registrar a chegada do carro, disponibilizar os assentos e controlar as condições necessárias para partida.	OK	Não teve falha	Na implementação da lógica de chegada do carro e controle das vagas.	Commit d91e168.
5	S1	Implementei station_wait_for_car para fazer os passageiros aguardarem a chegada do carro e reservarem os assentos disponíveis.	OK	Não teve falha	Depois de implementar a chegada do carro, no desenvolvimento da lógica de espera e reserva de assentos.	Commit 885bfdc.

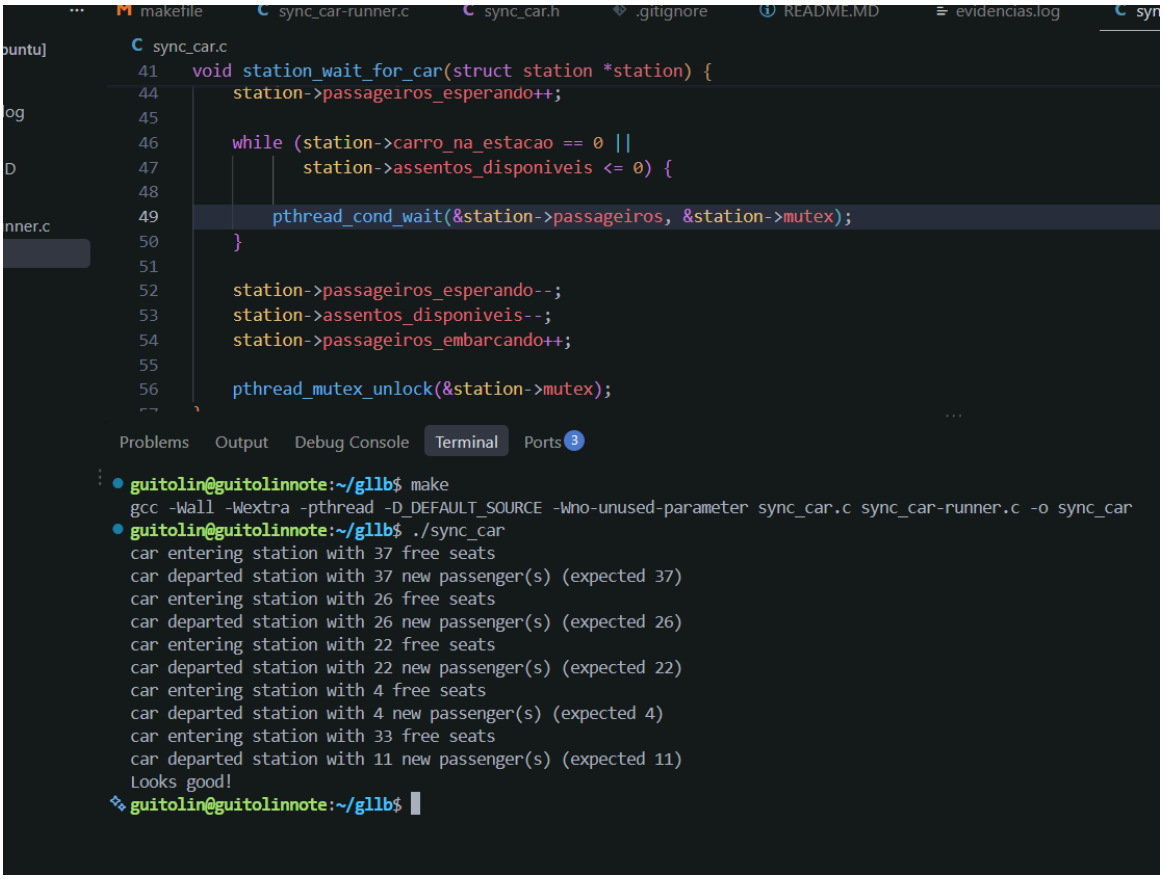
6	S1	Implementei station_on_board para registrar quando cada passageiro termina de embarcar e atualizar o controle de passageiros que faltam.	OK	Não teve falha	Depois de implementar a espera dos passageiros, durante o desenvolvimento da confirmação do embarque.	Commit 37f2394.
7	S2	Corrigi a sincronização entre as threads para melhorar o controle das condições de embarque e garantir que o carro esperasse as confirmações antes de partir.	OK	Eu precisei revisar as threads pra garantir que o carro não saísse enquanto ainda tivesse passageiros sem ter embarcado..	Na parte de revisão, depois de desenvolver as funções principais.	Commit 10ed73b.
8	S3	Compilei e executei o programa utilizando o teste fornecido pelo professor para verificar se a implementação atendia ao comportamento esperado.	OK	Não teve falha	depois de concluir os ajustes, durante a compilação e execução dos testes	Saída do teste no terminal, mostrando “Looks good!.”

Prints

• Erro principal: o momento do bug mais relevante da semana.

durante o desenvolvimento não ocorreram bugs ou erros relevantes que exigissem uma correção específica. Por isso, não tem um print de erro para apresentar. Como evidência da validação, eu registrei o teste final fornecido na atividade, que foi executado com sucesso e retornou Looks Good!. Os únicos erros e problemas que eu enfrentei foram por enviar comandos errados sem querer no terminal e por erros de escrita que foram rapidamente corrigidos.

• Validação final: teste passando / saída correta.



7. Uso de IA

Onde usei IA (2–3 linhas): Usei a IA pra me ajudar a organizar a ordem em que eu deveria fazer os commits e desenvolver a atividade. Também pedi ajuda para lembrar alguns comandos do terminal que eu tinha esquecido. Além disso, pedi para a IA fazer um resumo do enunciado de uma forma mais simples e didática, para que eu conseguisse entender melhor o que precisava fazer. Também usei a ferramenta para entender a lógica da função station_load_car, que foi uma das partes que achei mais difíceis no começo.

Prompts principais (liste, sem história):

1. Organização dos commits: Pedi à IA para analisar as etapas da atividade e me ajudar a organizar a ordem em que deveria desenvolver cada parte e fazer os commits no GitHub, mantendo as alterações sem divididas.
2. Entendimento do enunciado: Pedi à IA para fazer um resumo do enunciado com uma linguagem mais simples e didática, explicando o objetivo da atividade, o que cada função deveria fazer e quais requisitos precisavam ser seguidos.
3. Comandos do terminal: Pedi ajuda para lembrar os comandos necessários para compilar, executar e testar o programa, além de entender a ordem em que deveria utilizá-los.
4. Lógica da função station_load_car: Pedi à IA para explicar passo a passo o funcionamento da função, principalmente a lógica de sincronização entre o carro e os passageiros, pois tive dificuldade para entender essa parte no início.

O que validei manualmente e como: Fiz o Makefile para compilação e implementei a struct station. Também rodei os testes para conferir se o programa estava funcionando corretamente, iniciei a gravação do script e salvei os comandos utilizados. Além disso, preparei um readme completo e detalhado que explica melhor a atividade e adicionei o .gitignore para organizar os arquivos que não deveriam ser enviados para o GitHub.

8. Reflexão Final (5–8 linhas)

Reflexão: Durante a atividade eu consegui entender bem como funciona concorrência. No começo, eu tive um pouco de dificuldade para entender a lógica da sincronização entre o carro e os passageiros na função station_load_car, mas depois consegui aprender como funcionava. Também aprendi sobre mutex e variáveis de condição para a comunicação entre as threads. No final eu consegui implementar o que foi pedido e rodar os testes da atividade corretamente. A atividade foi interessante e um pouco mais fácil de conseguir entender a lógica do que as anteriores, na minha opinião, e isso ajudou bastante pra conseguir fazer essa entrega e para entender melhor os assuntos

9. Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Se tivesse mais 2 horas eu faria testes extras para verificar situações como um carro chegando sem passageiros esperando e a chegada de mais passageiros do que o número de assentos disponíveis. Também testaria a chegada de passageiros enquanto outro carro ainda ta na estação. Eu também iria revisar o código completo para ver se existem partes que poderiam ser melhoradas e se todas as partes de sincronização estão funcionando corretamente, eu também iria analisar o makefile para ver se a compilação continua funcionando sem warnings. E talvez, se desse tempo, eu detalharia ainda mais o readme para deixar mais fácil de entender a atividade e como ela funciona.